Program 2 - Scanner/parser

For this assignment, I used AI generated code as a starting point for several functions, then tested the code, made changes, and added additional cases to meet the assignment requirements. The following sections show code that I incorporated into my final program, including scanner, parser functions, and testing.

Scanner and Token Functions: Helped me create the token structure and scanner functions. These functions break the source code into token, identify number, string identifier and symbols, and keep track of line number. :

```
(struct token (type value line) #:transparent)

(define (show-tokens tokens)
  (for ([current-token tokens])
    (printf "Type: ~a | Value: ~a | Line: ~a\n"
          (token-type current-token)
          (token-value current-token)
          (token-line current-token))))




(define (string-index text character)
  (for/first ([i (in-range (string-length text))]
           #:when (char=? (string-ref text i) character))
    i))




(define (scan source)
  (define pattern

#px"^[0-9]+\\.[0-9]+|^[0-9]+|^[a-zA-Z][a-zA-Z0-9]*|^(=>|>=|<=|==|/=)|^[+*/;{}()\\[\\],:<>~=-]")

  (let loop ([remaining source]
          [line 1]
```

```
      [tokens '()])
(cond
 [(string=? remaining "")
  (reverse tokens)]

 [(char-whitespace? (string-ref remaining 0))
  (loop (substring remaining 1)
      (if (char=? (string-ref remaining 0) #\newline)
        (add1 line)
        line)
      tokens)]

 [(string-prefix? remaining "--")
  (define end (or (string-index remaining #\newline)
                  (string-length remaining)))
  (loop (substring remaining end) line tokens)]

 [(char=? (string-ref remaining 0) #\`)
  (define end
    (string-index (substring remaining 1) #\`))
  (if end
      (let ([value (substring remaining 0 (+ end 2))])
        (loop (substring remaining (+ end 2))
              line
              (cons (token 'string value line) tokens)))
      (error 'scan
             (format "Syntax error on line ~a" line)))]

 [else
  (define found (regexp-match pattern remaining))
  (if found
      (let* ([value (first found)]
             [type
              (cond
                [(regexp-match? #px"^[0-9]+\\.[0-9]+$" value)
                 'real]
                [(regexp-match? #px"^[0-9]+$" value)
                 'integer]
                [(regexp-match? #px"^[a-zA-Z]" value)
                 'identifier]
```

```
            [else 'symbol])])
        (loop (substring remaining (string-length value))
            line
            (cons (token type value line) tokens)))
        (error 'scan
            (format "Syntax error on line ~a" line)))])))
```

Parser Setup :

```
(define tokens '())
(define position 0)

(define (current-token)
  (if (< position (length tokens))
     (list-ref tokens position)
     (token 'eof "EOF"
          (if (null? tokens)
             1
             (token-line (last tokens))))))

(define (consume expected)
  (define current (current-token))
  (if (string=? (token-value current) expected)
     (begin
       (set! position (add1 position))
       current)
     (error 'parse
          (format "Syntax error on line ~a: expected ~a"
               (token-line current)
               expected))))
```

Expression parsing:

```
(define (parse-term)
  (let loop ([left (parse-unary)])
```

```
  (define op (token-value (current-token)))
  (if (member op '("*" "/" "and"))
    (let ([operator (consume op)]
          [right (parse-unary)])
      (loop (list (token-value operator) left right)))
    left)))

(define (parse-simple-expr)
 (let loop ([left (parse-term)])
  (define op (token-value (current-token)))
  (if (member op '("+" "-" "or" "~"))
    (let ([operator (consume op)]
          [right (parse-term)])
      (loop (list (token-value operator) left right)))
    left)))
```

Main parser:

```
(define (parse-program)
 (consume "program")
 (define name (parse-identifier))
 (consume "{")

 (define declarations (parse-var-declarations))
 (define routines (parse-routines))
 (define statements (parse-statement-list))

 (consume "}")

 (unless (eq? (token-type (current-token)) 'eof)
  (error 'parse
       (format "Syntax error on line ~a: unexpected token"
            (token-line (current-token)))))

 (list 'program name declarations routines statements))
```